SPELLCASTER

WIZARD DUEL | Game Design & Technical Feasibility Report

Two devices. Two wizards. One magical arena. No conventional game controller.

Prepared as a decision document for a fairy-tale-themed hackathon, with a focus on winning a Most Entertaining award through judge participation and a reliable short demo.

PROJECT AT A GLANCE	PROPOSED TARGET
Format	Head-to-head, real-time 1v1 web game
Input	Webcam hand gestures; optional spoken incantations
Session	60–90-second round; setup under ~30 seconds after permissions
Delivery	Browser-based; separate laptops; spectator/projector view
Core MVP	Room join, 3 spells, defenses, HP, timer, clear winner
Assumption	24–36-hour event, team of 3–4 developers

Executive summary

SpellCaster: Wizard Duel is a browser-based player-versus-player fairy-tale game in which each participant becomes a wizard by performing physical hand movements in front of a webcam. A compact real-time server validates spells and synchronizes a shared battle arena on two devices. Spoken incantations make the experience theatrical but should not be mandatory for the initial release.

The entertainment hook is the real-world performance: two judges visibly casting, shielding, countering, and competing. A signature spell-clash moment—in which simultaneous attacks collide in the center of the arena—provides the visual climax. The product should prioritize instant comprehensibility, generous timing, short matches, and a polished spectator view over complex character movement or artificial intelligence.

Key recommendation: Build a deterministic 2D duel with local gesture detection and a server-authoritative health/cooldown system. Launch with three dependable gesture interactions and treat voice as an enhancement.

1. Design goals and constraints

Goal	Practical design decision
Entertain immediately	Show a visible spell within the first 10–15 seconds of interaction.
Feature real physical interaction	Gestures trigger game actions; keyboard/mouse only for menus and fallback.
Support live judges	Two independent browser clients can join with one short code.
Be understandable at a distance	Large characters, readable HP bars, strong color/shape distinction, obvious hit reactions.
Avoid demo fragility	One-minute round, guided calibration, forgiving recognition, reliable fallback controls.
Fit short build cycle	Single arena, no accounts, no database, no video streaming, 3 core spells.

2. Player journey and game loop

2.1 Lobby and onboarding

1. Player A visits the app, chooses Create Duel, and receives a short room code and invite URL.
2. Player B opens the link on a different laptop and joins the room. Both pages request webcam permission; microphone permission is optional.
3. Each device displays a mirrored camera preview, a hand-detected status, and a three-gesture calibration: fist, open palm, two raised fingers.
4. Both players select Ready. The shared arena opens with a three-second countdown. One screen can additionally show a spectator-friendly layout.

2.2 Battle rules

- Each wizard begins with 100 HP. A round lasts 60 seconds by default; optionally 90 seconds for normal play.
- Players remain fixed on opposite sides of a 2D arena; the challenge is timing spells, anticipating counters, and managing cooldowns—not moving an avatar.
- Successful damage and well-timed defense charge an ultimate meter. The demo can use a scripted late-round boost to guarantee an exciting climax, but competitive mode should not manipulate outcomes.
- At 0 HP, the defeated wizard undergoes a comedic transformation. If time expires, the player with higher HP wins; tied HP triggers a short sudden-death phase.
- Each cast has visible telegraphing, travel time, hit feedback, and an audio cue. There is no frame-perfect input requirement.

2.3 Example interaction

Player A holds a fist to charge and opens the hand to send a fireball. Both screens immediately display the cast. Player B has roughly 0.8 seconds to hold an open palm and form a shield. The shield blocks the damage, charges the ultimate slightly, and breaks into sparkling fragments. Player B responds with a two-finger ice spell, briefly disabling A's attack input. Both players then cast simultaneously, triggering a spell clash that they must resolve with a sustained hand pose.

3. Spell system and balancing

Spell / action	Recognition / trigger	Effect	Starting balance
Incendio / Fireball	Hold fist ~0.3 s; release to open palm	Medium-damage projectile	20 dmg; 3 s cooldown
Protego / Shield	Hold open palm while not charging	Blocks a timed incoming strike	~0.9 s guard; 5 s cooldown
Glacius / Ice	Two fingers held steadily ~0.4 s	Light damage and short disable	10 dmg; 1.2 s lock; 7 s cooldown
Ultimate / Fulmen	Both hands raised; sufficient ultimate charge	Telegraphed heavy attack	30 dmg; meter-gated
Spell clash	Projectiles launched within a brief overlap window	Shared central tug-of-war sequence	Optional MVP+ feature

Values above are starting hypotheses requiring playtesting. Avoid long crowd-control durations or unstoppable repeated shields. Defense should create decisions: activating Protego too early leaves the player open when its short window ends.

3.1 Gesture state machine

A gesture should represent an intentional event, not an instantaneous classifier label. For Incendio: IDLE → FIST_STABLE → CHARGING → OPEN_PALM_RELEASE → CAST → COOLDOWN. Require a stable pose for several frames, add hysteresis for pose changes, and suppress repeated casts until the hand returns to a neutral state. Show the current recognized input onscreen for debugging and player confidence.

3.2 Voice design

Recommended: gestures cast base spells; a correctly spoken incantation within a short time window offers a small bonus such as enhanced particles or modest extra damage. Speech recognition must not be required to block or attack in demo mode. Two nearby microphones can hear both players, and venue noise may cause missed or duplicate commands. Later versions can use on-device keyword spotting, calibrated microphones, or headphones.

3.3 Spell clashes: the signature moment

If two offensive spells enter the center of the arena within a server-defined collision window, replace both projectiles with a large shared beam clash. Each player must hold their palm forward. The server accumulates validated hold progress over 2–3 seconds, perhaps with a slight advantage to the player who charged first. The winner's magic breaks through and delivers reduced damage. This mechanic is exciting to watch, but should be a post-MVP milestone: it adds synchronization, edge-case, and balance work.

4. Visual, audio, and fairy-tale direction

- Art direction: readable 2D storybook fantasy with an enchanted castle, stained-glass windows, and exaggerated wizard silhouettes. Avoid dependence on highly detailed 3D animation.
- Fairy-tale connection: apprentice wizards, cursed royal tournament, enchanted arenas, frog transformations, potion bottles, talking magical announcer, and kingdom-based rankings.
- Visual hierarchy: central duel, large health bars, incoming spell trajectory, clearly readable cooldown/meter icons, and a compact camera gesture indicator for each player.
- Audio: distinct cast/impact/shield sounds, announcer lines, and short musical stingers for clashes and victory. Mute voice input processing from the output path to reduce feedback.
- Comedy: a losing wizard transforms into a frog or teapot, an overconfident narrator comments on misses, and winning titles change with performance. Keep jokes short to preserve pacing.

5. Multiplayer system architecture

Each player's device runs camera processing locally and renders the same scene. Only discrete spell intentions and server-confirmed game events cross the network. The server is authoritative for HP, cooldowns, match clock, spell timing, victory conditions, and collision results. Clients are authoritative only for their local input observations, so this architecture is suitable for casual hackathon play rather than tamper-proof competitive esports.

Layer	Recommended technology	Responsibility
Frontend / lobby	React + Vite + TypeScript	Room join, device permissions, calibration, menus
2D presentation	Phaser	Characters, combat effects, animation, HUD
Gesture inference	MediaPipe Hand Landmarker / Gesture Recognizer	Local hand features and candidate poses
Voice (optional)	Web Speech API or local keyword	Incantation bonus; never a hard

	model	dependency
Audio output	Howler.js or Phaser audio	Sound feedback and soundtrack
Realtime service	Node.js + Socket.IO	Rooms, authoritative state, game events
Deployment	Static frontend hosting + persistent Node host	HTTPS frontend and wss-enabled realtime backend

5.1 Event protocol example

Client → server: JOIN_ROOM, READY, CAST_REQUEST(spell, clientActionId), DEFEND_START, DEFEND_END, CLASH_INPUT. Server → clients: ROOM_STATE, MATCH_START, SPELL_ACCEPTED, SPELL_REJECTED, DAMAGE_APPLIED, CLASH_START, CLASH_PROGRESS, MATCH_END. Every cast request should include a unique ID to prevent processing duplicates after retries.

5.2 Network and timing

- Use synchronized server timestamps for cooldowns and projectile outcomes; do not independently deduct damage on each client.
- Show a local cast animation immediately for responsiveness, but reconcile with a server acceptance event. Reject casts only when cooldowns or match state prohibit them.
- Make reaction windows forgiving, roughly 0.7–1.0 seconds at first, because network and camera processing add latency.
- Test two separate devices on the same Wi-Fi and on independent networks. HTTPS is generally necessary for browser camera/microphone access outside localhost.
- Reconnect handling should restore the player's slot and current game state for a short grace period, or display a clean rematch prompt.

5.3 Privacy

Camera frames and microphone audio should stay on each player's device in the MVP. Do not record, upload, or retain raw media. Tell players how their input is used. If browser speech recognition uses cloud processing, disclose that limitation or disable voice by default.

6. Demo plan: 90 seconds

Time	Action	What judges see
0–10 s	"No controller. You are the wizard." Invite two judges.	Immediate concept and challenge
10–25 s	Join using pre-opened links; cameras already calibrated for rehearsal.	Hand-tracking indicators, arena, countdown
25–40 s	Judge A casts a fireball; Judge B blocks.	Recognizable physical cause and effect
40–60 s	Ice counter + fast exchanges.	Competitive reaction and escalating tension
60–78 s	Both cast at once; trigger a spell clash.	Large central visual spectacle
78–90 s	Winner declared; loser turns into a frog.	Memorable payoff and short technical explanation

A demo preset should shorten cooldowns or precharge a clash if the time limit demands it, but should never silently fake the other player's gestures. Have a recorded video backup and a software-input fallback ready in case a device or network fails. Label those modes clearly.

6.1 Failure-proof demo checklist

- Two charged laptops, modern browsers, separate webcam permissions, stable camera placement, tested lighting, and a backup hotspot.
- Display a setup screen with clear tracking-confidence feedback; disable game start until both devices are ready.
- Pre-generate a room code and test rematch and refresh flow. Keep an operator-only reset control.
- Prepare short screen captures of a duel and a clash. If the live game fails, show the recording transparently rather than pretending it is live.
- Make sure audience can clearly see the shared arena without exposing distracting diagnostic interfaces.

7. Development plan

Phase	Deliverable	Acceptance test
0. Skeleton	Two browsers join a single room; arena and timer visible	Both clients show same room and countdown
1. Input	Reliable fist, palm, two-finger detection on one laptop	20 consecutive intentional gestures recognized with few false casts
2. Combat	Fireball, shield, HP, win conditions, cooldowns	Two players can complete an entire round
3. Polish	Impact VFX, audio, onboarding, loser transformation	A bystander understands attacks without explanation
4. Stretch	Voice bonuses, Glacius, ultimate, spell clash	Optional features cannot break the core duel
5. Rehearsal	Two-device network and noise tests	Three clean 90-second demonstrations back-to-back

7.1 Team ownership

- Developer A: MediaPipe input, gesture state machine, calibration, local debug overlays.
- Developer B: Phaser arena, spells, collisions/animations, audio, HUD, victory sequence.
- Developer C: Node/Socket.IO rooms, state transitions, damage/cooldowns, reconnection.
- Developer D (if available): UX, voice enhancements, integrated testing, deployment, judge demo.

Integration is a major schedule risk. Agree on spell names, action events, shared state shape, and visual coordinate conventions before building in parallel. The team should establish a playable end-to-end vertical slice before adding optional features.

8. Risks and mitigation

Risk	Impact	Mitigation
Gesture detection jitter	Accidental or missed casts	Stable-pose window, neutral reset, visible recognition, calibration
Microphone pickup / cross-talk	Wrong incantation or missed bonus	Voice optional; validate timing; per-device input
Network latency / desync	Contradictory HP or unfair blocks	Server-authoritative state; generous reaction windows
Complexity creep	No complete playable match	Freeze MVP at three actions and one arena
Browser permission friction	Awkward judge onboarding	Pre-tested HTTPS pages; permission status screen
Hardware/environment differences	Tracking inconsistency	Adequate lighting, adjustable camera angle, fallback input

Overpowered shield or freeze	Unfun combat	Short defensive windows, cooldowns, frequent balance testing
Spectator confusion	Weak entertainment payoff	Large visual cues, concise labels, central arena, strong audio

9. MVP vs. stretch scope

Must ship	Nice if time permits	Do not prioritize
2-player rooms	Incantation voice bonuses	Accounts / persistence
Three dependable gestures	Fourth ultimate spell	3D environments
Basic attacks / shield	Beam spell clashes	Complex free-roam movement
HP, clock, winner	Spectator URL / streamer overlay	Ranked matchmaking
One polished arena	Extra boss / arena themes	Generative AI dialogue
Calibration and demo mode	Comic announcer and achievements	Raw webcam streaming

10. Success criteria and go/no-go gates

- Gameplay: strangers can join and finish a meaningful head-to-head match without developer assistance after brief instructions.
- Reliability: intended spells register consistently under realistic lighting and venue noise; false casts are uncommon.
- Fairness: damage, cooldowns, and wins agree across both screens; no repeated actions from one held pose.
- Demo: first exciting event within ~15 seconds after start; at least one defensive response and one decisive payoff per round.
- Feasibility gate: if two-device casting is unstable near the midpoint of the event, cut voice, ice, and clash features until the core duel is reliable.

11. Decisions the team must make

Decision	Default recommendation	Why it matters
Match format	1v1 / fixed-position 2D duel	Best scope and readable gameplay
Primary casting	Gestures only; voice bonus	Keeps demo robust
Defensive input	Palm hold with short guard window	Easy to learn, hard enough to time
Spell count at launch	Fireball, shield, ice (or two attacks + shield)	Manageable gesture mapping
Spell clash	Stretch goal; special demo scene if stable	Signature moment with synchronization risk
Judge setup	Two laptops, one projector view	Avoid tiny split-screen demo
Branding	Original incantations optional	Avoid relying on a copyrighted franchise identity

12. Bottom-line recommendation

Proceed with SpellCaster: Wizard Duel if the team can deliver an end-to-end two-device fireball-versus-shield exchange early. That exchange is the proof of both technical viability and live entertainment value. Prioritize smooth input feedback, synchronized effects, short rounds, and a decisive fairy-tale payoff. Only add speech, extra spells, or beam clashes once the core duel can be demonstrated repeatedly without intervention.

Suggested pitch: “In fairy tales, magic lives in words and gestures. We made a game where it actually does. Challenge a friend, raise your hands, and duel—no controller required.”

Reference technologies (implementation starting points)

MediaPipe Gesture Recognizer for Web: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer/web_js

MediaPipe Hand Landmarker for Web: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/web_js

Socket.IO Rooms: https://socket.io/docs/v4/rooms/

Phaser: https://phaser.io/

Web Speech API compatibility/limitations: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

These are implementation references. Estimated timings, balance values, build effort, and judging potential in this document are design assumptions to validate through testing.